

A Natural $\mathcal{NP}$ -complete Problem

When we've proven that other problems L are $\mathcal{NP}$ -complete, we've done so by assuming that 3SAT is $\mathcal{NP}$ -complete, and then showing that $3\text{SAT} \leq_p L$. This is because it is easier to give one reduction than to show that $\forall A \in \mathcal{NP}, A \leq_p L$.

Question: *How can we show that $\forall A \in \mathcal{NP}, A \leq_p 3\text{SAT}$?*

Idea: We don't actually need to reduce from every possible A in $\mathcal{NP}$ – there's one language that “naturally” solves all of them!

The following problem is $\mathcal{NP}$ -complete:

$$A_{\text{B-NTM}} = \left\{ \langle M, w, \#^r \rangle \mid \begin{array}{l} M \text{ is a verifier that accepts some certificate } c \\ \text{of length at most } |w|^k \text{ for } w \text{ within } r \text{ steps} \end{array} \right\}$$

This problem is in $\mathcal{NP}$: the certificate is the c in the description.

A Natural $\mathcal{NP}$ -complete Problem (2)

To see why this has to be $\mathcal{NP}$ -hard, consider the example:

- $\text{HAM-PATH} \in \mathcal{NP}$, and a *yes*-instance $\langle G, s, t \rangle$ can be verified by a verifier M if we use a certificate P , where P is the Hamiltonian path. This process should take $\mathcal{O}(n^c)$ steps for some $c \in \mathbb{N}$.
- We can set $w = \langle G, s, t \rangle$ and r to be the worst-case time M can take for an input of size $|w|$.
- If we do so, then $\langle M, w, \#^r \rangle \in A_{\text{B-NTM}}$ iff $\langle G, s, t \rangle \in \text{HAM-PATH}$.

A similar argument will clearly work for any $A \in \mathcal{NP}$. So we really only need to show that $A_{\text{B-NTM}} \leq_p 3\text{SAT}$ – that is, we need to find a way to embed a TM operation into a boolean formula.

This is what the **Cook-Levin Theorem** does.

Building a Boolean Formula from a TM

We'd like to build a boolean formula ϕ that encodes the operation of a TM M . Let's start by looking at a deterministic TM, and then we can generalize to non-deterministic TMs or verifiers.

If M is deterministic, we can describe its operations by listing its configurations: e.g.,

$$C_0 = q_0 w_0 w_1 \dots w_{n-1}, C_1 = t_{0,1} q_1 w_1 \dots w_{n-1}, \dots$$

We can also stack its configurations like so:

$$\begin{array}{l} C_0 = q_0 w_0 w_1 \dots w_{n-1} \\ C_1 = t_{0,1} q_1 w_1 \dots w_{n-1} \\ \vdots \end{array}$$

We'll turn this stack into a table. Firstly, we note that there is some $k \in \mathbb{N}$ such that M cannot take more than n^k steps, since we're considering an M that halts in polynomial time (n^k is the r from the previous page).

Building a Boolean Formula from a TM (2)

We also note that, since M cannot move its head more than one square per step, it cannot look at more than n^k memory spaces. So we can buffer our stack with $\sqcup$ characters like so:

#	q_0	w_0	w_1	...	w_{n-1}	$\sqcup$	$\sqcup$	...	$\sqcup$	#
#	$t_{1,0}$	q_1	w_1	...	w_{n-1}	$\sqcup$	$\sqcup$	...	$\sqcup$	#
#										#
#										#
⋮	⋮	⋮	⋮							⋮
#	$t_{n^k-1,0}$	$t_{n^k-1,1}$	...						t_{n^k-1,n^k-1}	#

Building a Boolean Formula from a TM (3)

We have an $n^k \times (n^k + 2)$ table that completely describes the operation of M on w . We'll have to build a ϕ that encodes this M .

Question: *What are the properties of the table we need to encode into ϕ if we want it to emulate the TM operation?*

There are four major properties we care about:

1. Every cell contains exactly one character.
2. The characters on the first row match C_0 (it contains q_0 and the input).
3. There is a q_{accept} character somewhere on the table.
4. Every row in the table follows from the one above given the operation of M .

There are other properties we might want to encode, e.g., there is only one head character per row. But we can derive these properties from the four listed (e.g., from properties 2. and 4.).

So if we can build a ϕ encoding these four properties, we're done! (for deterministic TMs)

Building a Boolean Formula from a TM (4)

Since ϕ is already a conjunction of lots of smaller clauses, we can compose it of four different clause sets:

- ϕ_{cell} is a boolean formula that is true iff property 1. holds.
- ϕ_{start} is a boolean formula that is true iff property 2. holds.
- ϕ_{accept} is a boolean formula that is true iff property 3. holds.
- ϕ_{move} is a boolean formula that is true iff property 4. holds.

If we can build these formulas, then $\phi = \phi_{cell} \wedge \phi_{start} \wedge \phi_{accept} \wedge \phi_{move}$.

To start building these formulas, let's fix the variables we use: We say the variable $x_{i,j,s}$ is true if cell $[i, j]$ in the above table holds the character s .

Encoding ϕ_{cell}

To encode this property, we need to ensure that every cell contains at least one character that is true, but not more than one.

To ensure that at least one character is in cell $[i, j]$, we need to state that one of the character variables in that cell is true. So we write

$$\bigvee_s x_{i,j,s}.$$

To ensure that only one character is in the cell, we state that for any two characters s and s' , at least one of their character variables is false:

$$\bigwedge_{s,s', s \neq s'} (\overline{x_{i,j,s}} \vee \overline{x_{i,j,s'}}).$$

We can see that the second formula is already in 2CNF, while the first is a disjunction of many variables.

Encoding ϕ_{cell} (2)

We can turn a 2CNF clause into a 3CNF clause by repeating one of the literals (or adding a dummy literal that we force elsewhere to be false).

We can turn a disjunction of many variables into a 3CNF clause like so:

If the number of literals is four, we start with a clause such as $a \vee b \vee c \vee d$, and we separate it by adding new variables to get $(a \vee b \vee y_1) \wedge (\overline{y_1} \vee c \vee d)$.

So we can encode property 1. for a single cell at $[i, j]$ into a 3CNF formula. If we call this formula $\phi_{cell:i,j}$, then clearly the ϕ_{cell} we want is just

$$\phi_{cell} = \bigwedge_{i,j} \phi_{cell:i,j}.$$

And that's ϕ_{cell} .

To encode ϕ_{start} , we just manually set the characters on the first row to be C_0 :

$$x_{0,0,\#} \wedge x_{0,1,q_0} \wedge x_{0,2,w_0} \wedge \dots \wedge x_{0,n^k,\sqcup} \wedge x_{0,n^k+1,\#}$$

Every variable above is already in a 1CNF clause, so we can turn it into a 3CNF clause using the same trick as we used above.

And that's ϕ_{start} .

Encoding ϕ_{accept}

To encode ϕ_{accept} , we need to specify that the q_{accept} character can be found somewhere on the table. So we just write that one of its variables is true:

$$\bigvee_{i,j} x_{i,j,q_{accept}}.$$

We can turn this into a 3CNF using the trick described in the ϕ_{cell} section.

So we have ϕ_{accept} .

Encoding ϕ_{move}

This is the hard one. The trick is to remember (from assignment 1) that two neighbouring configurations can differ by only three characters. So if we look at any two adjacent rows in the table, we need to only worry about windows three characters long, e.g., in the table above,

#	q_0	w_0
#	$t_{1,1}$	q_1

q_0	w_0	w_1
$t_{1,1}$	q_1	w_1

$\square$	$\square$	$\square$
$\square$	$\square$	$\square$

Now, there may be many possible allowable windows of this sort – they just need to be consistent with the TM operation. So, for example, the

q_0	w_0	w_1
$t_{1,1}$	q_1	w_1

and

$\square$	$\square$	$\square$
$\square$	$\square$	$\square$

might occur in many places in the table.

Encoding ϕ_{move} (2)

We say a window is legal if, like the previous windows, it is consistent with the TM operation given the local knowledge we have in the window. So, for example, a window like

q_0	w_0	w_1
$t_{1,1}$	q_1	q_1

would not be legal, not just because it doesn't occur, but because it can't – the TM can't have two heads at once.

In order to ensure that the steps of the TM are encoded into ϕ , ϕ_{move} will have to check that every window is legal:

$$\phi_{move} = \bigwedge_{i,j} (\text{The } (i, j) \text{ window is legal}).$$

Note: If a TM accepts, we pretend that all of its subsequent configurations are just repeats of the accepting one.

Encoding ϕ_{move} (3)

Now, suppose that we write a window A at $[i, j]$ as

$A_{i,j}$	$A_{i,j+1}$	$A_{i,j+2}$
$A_{i+1,j}$	$A_{i+1,j+1}$	$A_{i+1,j+2}$

Then the property that the (i, j) window is legal can be encoded as:

$$\bigvee_{A: A \text{ is legal}} \left(\begin{array}{ccc} x_{i,j,A_{i,j}} & \wedge x_{i,j+1,A_{i,j+1}} & \wedge x_{i,j+2,A_{i,j+2}} \\ \wedge x_{i+1,j,A_{i+1,j}} & \wedge x_{i+1,j+1,A_{i+1,j+1}} & \wedge x_{i+1,j+2,A_{i+1,j+2}} \end{array} \right).$$

Encoding ϕ_{move} (4)

We note that because the A are just the 2×3 windows that could be consistent with the operation of M , the number of A is finite, and so the above formula exists – and is even bounded in size for all i and j .

We leave it as an exercise to show that this formula can be written in 3CNF, but we note that when we do, the size of the formula depends only on M , not on the input size. So we can get a 3CNF formula $\phi_{move:i,j}$.

Given $\phi_{move:i,j}$, we can just write $\phi_{move} = \bigwedge_{i,j} \phi_{move:i,j}$.

And that's ϕ_{move} .

Putting it all Together

Since we've defined ϕ_{cell} , ϕ_{start} , ϕ_{accept} , and ϕ_{move} , we also have $\phi = \phi_{cell} \wedge \phi_{start} \wedge \phi_{accept} \wedge \phi_{move}$, as we stated above.

The only problem that remains is to ensure that the reduction is polynomial in size.

But we note that ϕ_{cell} is just the same formula for all cells. It is $\mathcal{O}(n^{2k})$ in size. The same holds for ϕ_{accept} and ϕ_{move} (note that the individual $\phi_{move:i,j}$ may be very large, but it depends on M , not on the size n of w).

We can also see that ϕ_{start} has a clause for every cell in the first row of the table: it has size $\mathcal{O}(n^k)$.

So our entire reduction is of size $\mathcal{O}(n^{2k})$, and so the reduction is polynomial in time.

What about NTMs?

To finish off, we remember that we were assuming that M was deterministic. To make it non-deterministic we can either:

- Move the non-determinism to the beginning of the program execution by making M a verifier and including a certificate C in the input. If we do so, we simply leave the C undetermined when defining ϕ_{start} .

We've already mentioned this idea above.

- Add legal windows when defining ϕ_{move} to cover the possibility of non-determinism during the program run.

Either way, our reduction gives us a ϕ in polynomial time, and so

$$A_{B\text{-NTM}} \leq_p 3\text{SAT}.$$

So 3SAT is $\mathcal{NP}$ -complete, as desired.

And that finishes our section on the class $\mathcal{NP}$. Next time we'll start to look at space complexity.

Space Complexity

The space complexity of a problem or program is the amount of memory needed for it. In terms of TMs, this is the number of tape cells used.

We've already touched on this idea: If a TM takes at most k steps in its operation, then it also uses at most k memory cells – it can only look at one cell per step.

But a TM can also look at a single cell more than once. *So it is possible to re-use steps in space, but not time.*

Otherwise, the complexity classes based on space are similar to those based on time: if a TM M halts on all inputs of length n using at most $f(n)$ tape cells, it is a program that has a space complexity of $f(n)$.

The Classes $\mathcal{PS}$ and $\mathcal{NPS}$

This extends to definitions for languages:

Definition: $\text{SPACE}(f(n)) = \{L \mid L \text{ can be decided by a } \mathcal{O}(f(n))\text{-space TM.}\}$

Definition: $\text{NSPACE}(f(n)) = \{L \mid L \text{ can be decided by a } \mathcal{O}(f(n))\text{-space NTM.}\}$

As before, we'll look at the problems that can be solved or verified in an efficient (polynomial) amount of space:

Definition: $\mathcal{PS} = \bigcup_{k \in \mathbb{N}} \text{SPACE}(n^k)$.

Definition: $\mathcal{NPS} = \bigcup_{k \in \mathbb{N}} \text{NSPACE}(n^k)$.

Any language where the whole input is needed to decide membership takes at least $\mathcal{O}(n)$ space. Can you see why?

$$\mathcal{P} \subseteq \mathcal{PS} \subseteq \mathcal{NPS}$$

The big question we want to answer is how these complexity classes compare to the ones we already know – if we measure the space used instead of time, do we get any extra power?

Once again, the answer is that we're not sure. But it looks like we do get more power.

Firstly, we have seen that any TM that takes $\mathcal{O}(n^k)$ time also takes $\mathcal{O}(n^k)$ space.

So $\mathcal{P} \subseteq \mathcal{PS}$. Similarly, $\mathcal{NP} \subseteq \mathcal{NPS}$.

It is also clear that since any TM is also an NTM, that $\mathcal{PS} \subseteq \mathcal{NPS}$.

Idea: We can write the nondeterministic choices made by an NTM as a string (a certificate) — and we can loop over strings!

On input x :

1. For every certificate C :
2. Try to use C to verify x .
3. If C is a valid certificate for x , *accept*.
4. *Reject*.

The number of certificates is generally exponential, so this will generally take an exponential amount of time.

But the certificates are polynomial in size, and so we can loop over all of them in polynomial space (treat the strings as numbers and increment them by one every loop iteration). Since the inner loop requires only polynomially more space, the entire process takes a polynomial amount of space.

Savitch's Theorem: $\mathcal{PS} = \mathcal{NPS}$

Big Theorem:

Savitch's Theorem:

For any $f : \mathbb{N} \rightarrow \mathbb{R}^+$, $\text{NSPACE}(f(n)) \subseteq \text{SPACE}([f(n)]^2)$.

Since this blowup is polynomial in size, we the extra space needed to simulate an NTM on a deterministic TM will not be enough to make any problem that is polynomial in the non-deterministic space grow into a super-polynomial problem in deterministic space.

Question: *Why is this possible, and why doesn't it help us determine whether $\mathcal{P} \neq \mathcal{NP}$?*

High-Level Idea:

- The $\mathcal{NP} \subseteq \mathcal{PS}$ idea won't work, since the certificate is now exponentially sized.
- **But** we can store configurations instead of certificates. We can then ask, for two configurations C_i and C_j , whether C_j can be derived from C_i in at most t steps, for some $t \in \mathbb{N}$.
- Instead of actually running the NTM to generate configurations, we look for a midpoint configuration: We ask whether there exists a configuration C' such that C' can be derived from C_i in at most $t/2$ steps and C_j can be derived from C' in at most $t/2$ steps.
- We use recursion to turn this into something like a binary search.
- Once we have this idea, we just ask whether C_0 can generate an accepting configuration within $2^{O(n^k)}$ time.

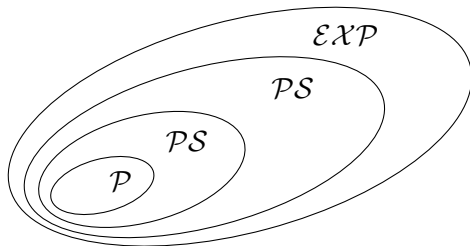
Since the search is exponential, it doesn't help with $\mathcal{P}$ versus $\mathcal{NP}$.

What we Know

$$\mathcal{P} \subseteq \mathcal{NP} \subseteq \mathcal{PS} = \mathcal{NPS} \subseteq \mathcal{EXP}$$

Note: $\mathcal{EXP}$ is just the exponential time analogue to $\mathcal{P}$.

None of the subset inclusions have yet been proven to be proper, but we know that at least one of them must be. *We strongly suspect that all of the classes are different:*



Reductions in $\mathcal{PS}$

We can use polytime reductions in relation to $\mathcal{PS}$ in the same way that we do when looking at $\mathcal{P}$ and $\mathcal{NP}$:

- If $A \leq_p B$ and $B \in \mathcal{PS}$, then $A \in \mathcal{PS}$.
- If $A \leq_p B$ and $A \notin \mathcal{PS}$, then $B \notin \mathcal{PS}$.

The ideas of class hardness and completeness carry over, as well:

A language L is $\mathcal{PS}$ -complete if:

- $L \in \mathcal{PS}$.
- $\forall A \in \mathcal{PS}, A \leq_p L$.

Reductions in $\mathcal{PS}$ (2)

Question: *Why do we use polytime reductions rather than polyspace reductions?*

We generally use reductions to show that a problem is hard – to show, e.g., that B is $\mathcal{NP}$ -complete, and so likely not in $\mathcal{P}$. This only works when the power we have for the reduction is limited.

If our reductions in $\mathcal{PS}$ were allowed to use polynomial space instead of time, we could solve any problem in $\mathcal{PS}$ as part of the reduction. This would make all non-trivial languages in $\mathcal{P}$ $\mathcal{PS}$ -complete!

A $\mathcal{PS}$ -complete problem

Just like in the class $\mathcal{NP}$, we know a $\mathcal{PS}$ -complete language: the True Quantified Boolean Formulas (TQBF):

$$\text{TQBF} = \{ \langle \phi \rangle \mid \phi \text{ is a true fully quantified boolean formula.} \}$$

A *fully quantified* boolean formula is a ϕ just like the ones in SAT, except we have no limits to the number of quantifiers: e.g.,

$$\phi = \forall x, y, \exists z, (x \vee z) \wedge (\bar{y} \vee z).$$

The proof that this is $\mathcal{PS}$ -complete is somewhat similar to the proof of the Cook-Levin theorem that we saw last week, with a space-saving mechanism a little bit like the one used in the proof of Savitch's theorem.

You can think of an $\mathcal{NP}$ problem like SAT or 3SAT as a problem in which there is only a single existential quantifier “ $\exists$ ” that covers all of the variables. Similarly, co- $\mathcal{NP}$ uses a single universal quantifier “ $\forall$ ”. This is a generalization.

Why $\mathcal{PS}$ is Interesting

We can treat choices made under quantifiers as being made by adversaries in a game:

- If I say that there is an input x that makes something like a boolean formula ϕ true ($\exists$), I'm saying I could tell you that x , so long as I can find it.
- If I say that for all inputs y , ϕ true ($\forall$), I'm saying that you can't tell me an input y that makes that ϕ false.
- If I chain many quantifiers together, we can take turns choosing inputs – I choose the ones covered by $\exists$ and you choose the ones covered by $\forall$. If I say the ϕ is true with these quantifiers, what I'm saying is that even if you try to make the ϕ false, I can always make choices that force the end result to be true.

Why $\mathcal{PS}$ is Interesting (2)

- If we were playing a game in which I was trying to make ϕ true while you were trying to make it false, and if I could always manage this, I'd be saying that I had a winning strategy.
- When we look at it this way, $\mathcal{PS}$ -complete problems start to look like games – we take turns making moves, and we each try to reach our (mutually exclusive) winning conditions. So it'll not be too much of a surprise that a number of generalizations of two-player games are $\mathcal{PS}$ -hard.

This is why $\mathcal{PS}$ is important – $\mathcal{PS}$ -completeness comes up when we're working in areas in which a program (like a robot) is competing against another agent. It also comes up when the robot is repeatedly taking measurements of its surroundings – our worst-case analysis treats the surroundings like an adversary, even if there is no actual adversary present.

An Example $\mathcal{PS}$ -complete Game

We've said that many generalizations of games are $\mathcal{PS}$ -hard. These games include Hex, Reversi, and some versions of Go (when these games are played on arbitrarily large boards).

We should note that several one-player games such as Sokoban also have $\mathcal{PS}$ -complete extensions.

To show these games are complete, we need to reduce to them from another $\mathcal{PS}$ -complete problem. We'll give an example of one such reduction to a simple game: the generalized geography game.

Generalized Geography

When playing *Geography*, players take turns naming cities. There are two rules:

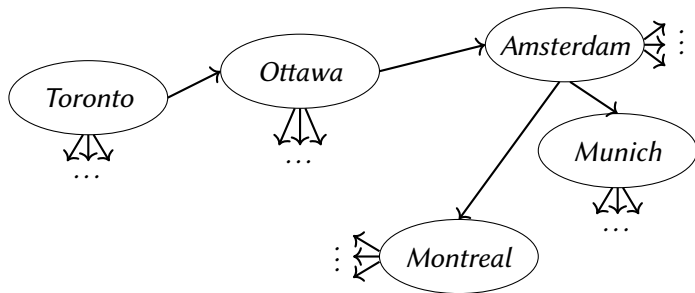
1. No city can be named more than once.
2. The last letter of the current city named must be the first letter of the next.

E.g., *Toronto, Ottawa, Amsterdam, Munich,*

The first player who cannot continue the chain loses.

Generalized Geography (2)

We can express this game by putting different city names on a graph:



That is, we draw an arrow between nodes u and v if the last letter of the city name in u is the first letter in v .

Generalized Geography (3)

When using this graph to play geography, each player takes turns choosing the next node to visit: the next node must be reachable from the current one using a single edge, and no node can be visited twice.

Generalized Geography is the same, except we can play it on any directed graph G . In order to argue that this game is $\mathcal{PS}$ -complete, we treat it as a language. So we formulate the game as:

$$\text{GG} = \left\{ \langle G, s \rangle \mid \begin{array}{l} \text{Player A has a winning strategy for generalized} \\ \text{geography using } s \text{ as the starting vertex.} \end{array} \right\}$$

Generalized Geography is in $\mathcal{PS}$

We can solve it using the following code:

$M =$ “On input $\langle G, s \rangle$:

1. If s has out-degree zero, reject, since player A loses.
2. Remove s and all edges containing s .
3. For each of the nodes $v_1, v_2, \dots, v_k$ that s originally pointed to, recursively call M on $\langle G, v_i \rangle$.
4. If all recursive calls accept, player B has a winning strategy, so *reject*. Otherwise, player B doesn't have a winning strategy so player A must be able to force a win. So *accept*.

Question: How much space is required for this program?

There are at most ($n =$ the number of nodes in G) levels of recursion, and each step in the recursion just requires us to store a subgraph of G . So this is in $\mathcal{PS}$.

Generalized Geography is $\mathcal{PS}$ -hard

So to show that this is $\mathcal{PS}$ -complete, we just need to show that $\text{TQBF} \leq_p \text{GG}$.

Recall that this means that we want to be able to use GG to solve TQBF – if we have a quantified boolean formula ϕ we'd like to build, in polynomial time, a corresponding graph G with a starting node s . If we do it right, then determining if G and s are in GG would tell us whether ϕ is in TQBF.

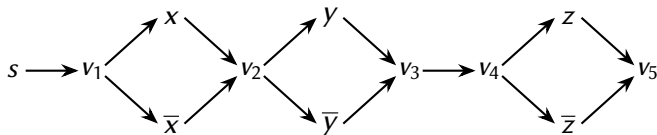
Let's have a look at the ϕ from the previous class:

$$\phi = \forall x, y, \exists z, (x \vee z) \wedge (\bar{y} \vee z).$$

How do we Start?

Quantifiers are a bit like player choices – one player chooses the variables under the $\forall$ quantifiers and tries to make the formula false, while another chooses the variables under the $\exists$ quantifiers and tries to make it true. We say that player A is trying to make the formula true.

So player B chooses x and y , and player A chooses z . A good place to start building our full graph would be to build a graph that makes the players go through a similar choice. One possible way to do this is to create nodes x , $\bar{x}$, and so on, in the graph, and connect them like so:

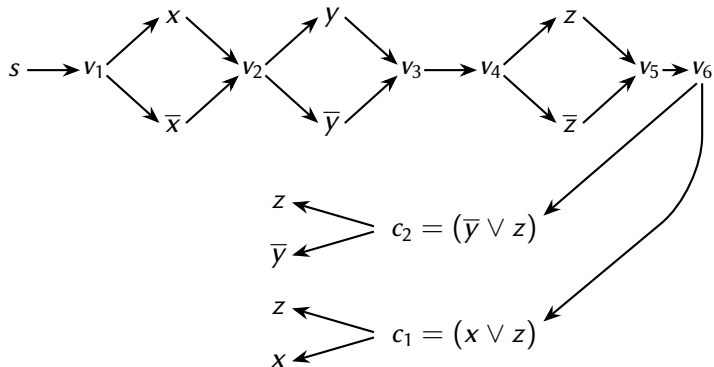


What about the rest of the formula?

The variable choices have been made, now the players want to show the whole thing is true (or false).

- We don't actually check the whole formula — we check only one clause, term, literal, and so on.
- If the formula is $(\text{clause } 1) \wedge (\text{clause } 2) \wedge \dots \wedge (\text{clause } m)$, the formula is true only if all clauses are true, but false if one is false.
- Player B is the one trying to make everything false, and that's what could be done here in a single check. So player B chooses the clause.
- If the formula is $(\text{term } 1) \vee (\text{term } 2) \vee \dots \vee (\text{term } m)$, the formula is false only if all clauses are false, but true if one is true.
- Player A is the one trying to make everything A, and that's what could be done here in a single check. So player A chooses the term.
- If we see a negation, the players switch roles.

The Resulting Graph



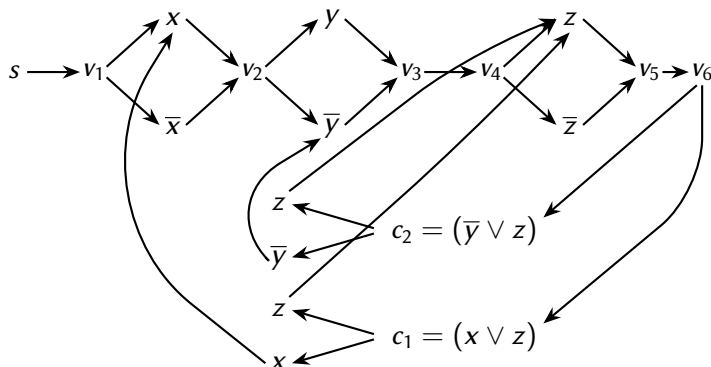
The Resulting Graph (2)

we've added nodes to the graph for every clause and subclause in the formula. If the clauses are joined by *ands*, player B chooses which clause to go to. If they are joined by *ors*, player A chooses. If there is a *not*, the role of the players changes – player A wants to make the clause false. So we put another step in the graph so make the players switch turns.

Finally, we want to finish the game. We should do this in a way such that if player A wins if the final literal is true, and player B wins otherwise (aside from the role switching through *nots* and keeping in mind who is playing at each node).

We do this by connecting the endpoints of the graph back to the nodes in the variable choices so that the appropriate player wins.

The Final Graph



And that's the reduction! We can see that the same idea will work for any formula ϕ , and that the construction is polytime in ϕ . So $\text{TQBF} \leq_p \text{GG}$, and so GG is $\mathcal{PS}$ -complete.

Logarithmic Space

The other space complexity class we'll talk about is the class of logarithmic space problems – that is, the class of problems that take an amount of space that's logarithmic in the size of the input.

Since the input already takes $\mathcal{O}(n)$ space, though, we can't include it in the memory being counted. We consider only TMs that have a read-only input tape, a logarithmic sized work tape, and a write-only output tape. (The inputs and outputs have to be restricted, otherwise we could cheat and get a linear amount of memory).

Definition: $\mathcal{L} = \{L \mid L \text{ is decided by a TM that operates in } \mathcal{O}(\log n) \text{ space}\}.$

Definition: $\mathcal{NL} = \{L \mid L \text{ is decided by an NTM that operates in } \mathcal{O}(\log n) \text{ space}\}.$

Important: this says $\mathcal{O}(\log n)$, not $\mathcal{O}(\log^k n)$.

Why do we Care About this Class?

The idea is that we might be running a machine M that is itself small (limited memory), but which has access to a very large database D . In order to access D , M must be able to say where on D it wants to look. So if D has 2^n entries, M must be able to store $\log(2^n) = n$ numbers.

This type of application comes up when, say, M is your phone or laptop and D is the Internet.

It's because we're looking at this sort of application that we also have the restriction that we work in a space that's $\mathcal{O} \log n$ and not, say, $\log^2(n)$. (Also because $\log(n^k) = k \log n$).

Logspace, Continued

This is a very restricted class – there's really not much we know how to do when we can't even copy the input to the working tape. But we can do some things: we can add and multiply numbers, and we can decide simple languages like $\{0^n 1^n \mid n \in \mathbb{N}\}$.

How to decide $\{0^n 1^n \mid n \in \mathbb{N}\}$ in logspace: we can't write to the input tape or copy the input. We can count the zeros and ones, since the space needed to remember a number n is logarithmic in n .

On input x :

1. Set a counter c to 0.
2. While the tape head reads a 0:
3. Increment c by 1 and move the head right.
4. While the tape head reads a 1:
5. Decrement c by 1 and move the head right.
6. If c is set to 0 as the end of the input is reached, *accept*. If it reaches 0 before that time, or if it is non-zero at the end of the input, *reject*.

Savitch's Theorem Doesn't Apply Here

Reason: $\mathcal{L}$ uses $\mathcal{O} \log n$ and not $\mathcal{O} \log^2(n)$ space is that Savitch's theorem doesn't give us a way to emulate non-determinism and still stay in the class. So the best we can currently say is that $\mathcal{L} \subseteq \mathcal{NL}$. We believe that the two classes are not equal.

So what we know is that

$$\mathcal{L} \subseteq \mathcal{NP} \subseteq \mathcal{P} \subseteq \mathcal{NP} \subseteq \mathcal{PS} \subseteq \mathcal{EXP}.$$

We believe all of the inequalities are strict, but aside from the facts that $\mathcal{L} \neq \mathcal{PS}$ and $\mathcal{P} \neq \mathcal{EXP}$, we don't yet have any proofs. We haven't even proven that $3\text{SAT} \notin \mathcal{NL}$!

Logspace Reductions

Just like polyspace reductions are too powerful for $\mathcal{PS}$ analysis, polytime reductions are too powerful for logspace analysis.

Definition: We say that $A \leq_L B$ if there is a function f computable in logspace such that $x \in A$ iff $f(x) \in B$.

If we're working in logspace we won't, in general, even have the space to write $f(x)$. But the point of reductions is to argue that B can be used to solve A . So when testing whether $f(x)$ is a member of B in order to tell us whether x is in A , we've got to test whether $f(x) \in B$. When we do so, we have to call the function f repeatedly.

Computations in $\mathcal{L}$ and $\mathcal{NL}$ often involve repeating the same calculation over and over in order to save space.

Definition: A language L is $\mathcal{NL}$ -complete if it is in $\mathcal{NL}$, and if for every language A in $\mathcal{NL}$, $A \leq_L L$.

PATH is $\mathcal{NL}$ -complete

Our go-to $\mathcal{NL}$ -complete language is PATH.

The language PATH is clearly in $\mathcal{NL}$: you can just start at s and non-deterministically follow edges until you reach t . You can halt after checking n vertices, where n is the number of vertices in G .

The proof that $\forall A \in \mathcal{NL}, A \leq_L \text{PATH}$ is more complicated. Like Savitch's theorem, we won't cover this in detail, and we won't be expecting you to remember this proof, but it is good to know. You can see Sipser 8.25 for details.

Finally, we can show that $\text{PATH} \in \text{co-}\mathcal{NL}$! Again, we won't show the proof, but you can find it in Sipser 8.27.

What this means is that $\mathcal{NL} = \text{co-}\mathcal{NL}$.